

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Bases de datos distribuidas				
NÚMERO DE LA PRÁCTICA:	08	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	09/06/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Barrios Medina, Mathias Alonso - Hanco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea - Pacheco Palo, Fabiana Francinet - Quispe Condori, Alvaro Raul				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
I. DESARROLLO DE EJERCICIOS PROPUESTOS Repositorio del laboratorio: https://github.com/ST0731-29-3821-3041/SD_labSQL Caso de estudio: FarmaAndes S.A. FarmaAndes S.A. es una cadena farmacéutica peruana con centros de distribución en Arequipa, Lima y Cusco. Cada sede posee una base de datos local para controlar su inventario. Cuando una sucursal solicita medicamentos a otra sede, el sistema debe realizar una transacción distribuida para garantizar que: 1. El inventario se descuenta del almacén origen. 2. El inventario se incrementa en el almacén destino. 3. Ambas operaciones sean atómicas. Si una operación falla, toda la transacción debe revertirse. Implementación: Para comenzar con la implementación, lo primero que se hizo fue crear las bases de datos para cada sede: <code>almacen_arequipa</code> y <code>almacen_lima</code>. <pre>CREATE DATABASE almacen_arequipa; CREATE DATABASE almacen_lima;</pre>

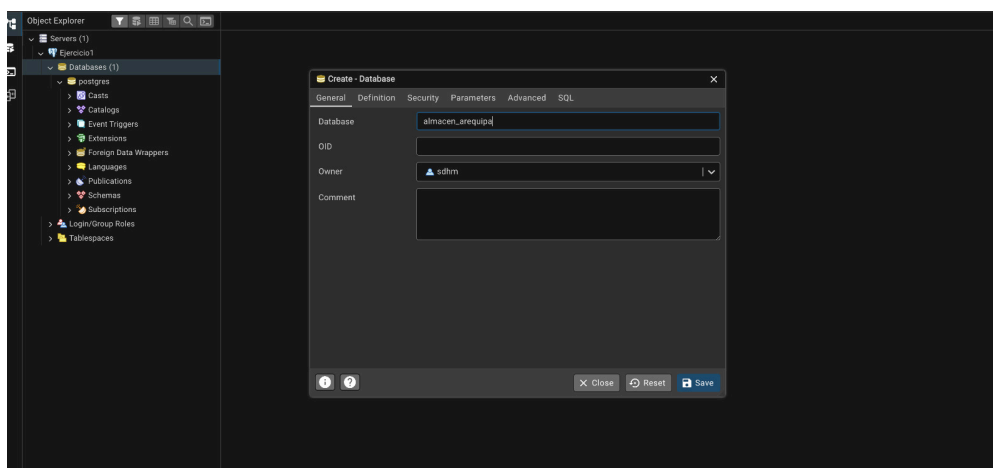


Figura 1: Creación de la base de datos almacen_arequipa

Luego se creó la tabla inventario en ambas bases de datos. Para Arequipa los valores iniciales fueron 100 unidades de Paracetamol, 80 de Ibuprofeno y 50 de Amoxicilina.

```
CREATE TABLE inventario(
  id serial PRIMARY KEY,
  producto VARCHAR(100),
  stock INTEGER
);

INSERT INTO inventario(producto, stock)
VALUES
('Paracetamol', 100),
('Ibuprofeno', 80),
('Amoxicilina', 50);
```

Para Lima se utilizó la misma estructura de tabla, pero con valores iniciales distintos: 50 de Paracetamol, 30 de Ibuprofeno y 20 de Amoxicilina.

```
CREATE TABLE inventario(
  id serial PRIMARY KEY,
  producto VARCHAR(100),
  stock INTEGER
);

INSERT INTO inventario(producto, stock)
VALUES
('Paracetamol', 50),
('Ibuprofeno', 30),
('Amoxicilina', 20);
```

Las siguientes capturas evidencian la creación de la tabla y el ingreso de datos en ambas sedes.

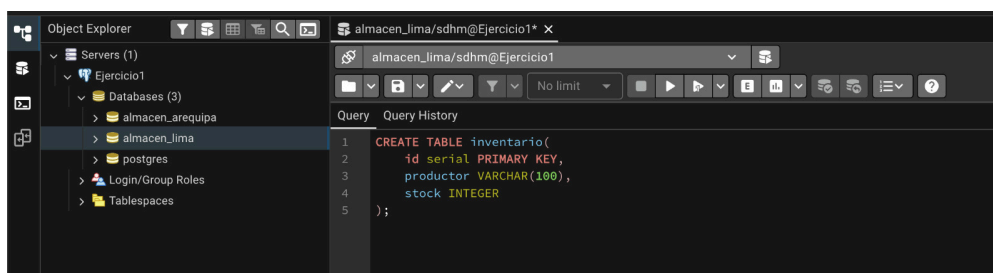


Figura 2: Creación de la tabla inventario en almacen_lima

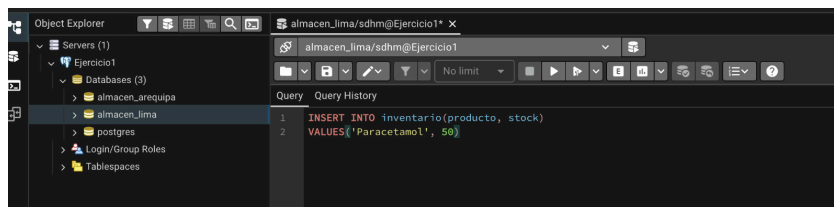


Figura 3: Inserción de datos en la tabla inventario de almacen_lima

El mismo procedimiento se repitió para Arequipa. A continuación se verifica que los datos se hayan registrado correctamente.

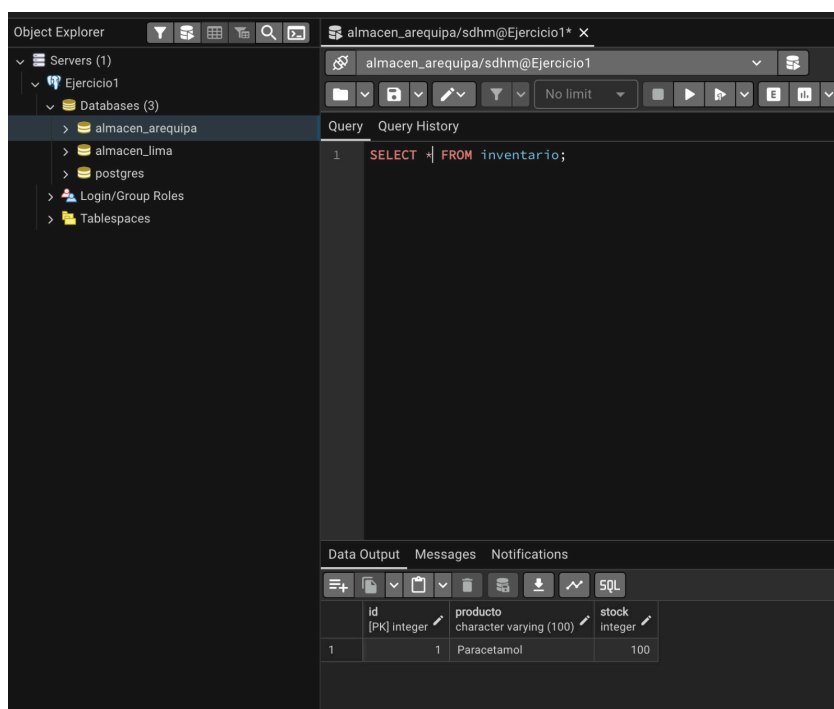


Figura 4: Verificación de datos ingresados en almacen_arequipa

Para la interacción con el usuario se desarrolló una interfaz web con Flask. El módulo db.py encapsula la conexión a PostgreSQL y app.py define las rutas para realizar transferencias y simular fallos. A continuación se muestra el código de app.py con comentarios que explican cada ruta.

```
# Aplicación web Flask – FarmaAndes S.A.
# Interfaz para ejecutar transferencias distribuidas con 2PC

from flask import Flask, render_template, request
from transaccion import (
    SEDES,
    obtener_inventario,
    obtener_productos,
    ejecutar_2pc
)

app = Flask(__name__, template_folder="../templates")

def renderizar_inicio(logs=None):
    """Renderiza la página principal con inventarios y logs de la última operación"""

    if logs is None:
        logs = []
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

return render_template(
    "index.html",
    productos=obtener_productos(),
    inventario_arequipa=obtener_inventario("almacen_arequipa"),
    inventario_lima=obtener_inventario("almacen_lima"),
    sedes=SEDES,
    logs=logs
)

def leer_formulario():
    """Extrae los datos del formulario HTML enviado por el usuario"""

    return {
        "producto": request.form["producto"],
        "cantidad": int(request.form["cantidad"]),
        "origen": request.form["origen"],
        "destino": request.form["destino"]
    }

# Ruta principal: muestra el estado inicial de los inventarios
@app.route("/")
def index():
    return renderizar_inicio()

# Ruta para transferencia exitosa (Ejercicio 1):
# Ejecuta 2PC con simular_fallo=False (comportamiento normal)
@app.route("/transferir", methods=["POST"])
def transferir():

    datos = leer_formulario()
    logs = ejecutar_2pc(
        datos["origen"],
        datos["destino"],
        datos["producto"],
        datos["cantidad"]
    )
    return renderizar_inicio(logs)

# Ruta para simulación de fallo (Ejercicio 2):
# Ejecuta 2PC con simular_fallo=True (el nodo destino "cae")
@app.route("/fallo", methods=["POST"])
def fallo():

    datos = leer_formulario()
    logs = ejecutar_2pc(
        datos["origen"],
        datos["destino"],
        datos["producto"],
        datos["cantidad"],
        simular_fallo=True
    )
    return renderizar_inicio(logs)

if __name__ == "__main__":
    app.run(debug=True)

```

La aplicación se ejecuta localmente y presenta una interfaz que permite seleccionar producto, cantidad, origen y destino.

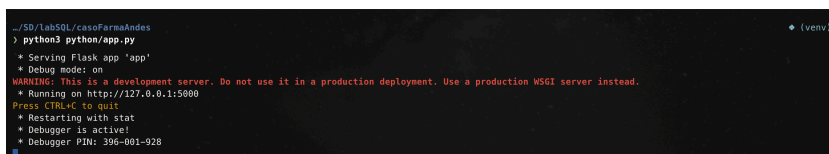


Figura 5: Ejecución de la aplicación Flask

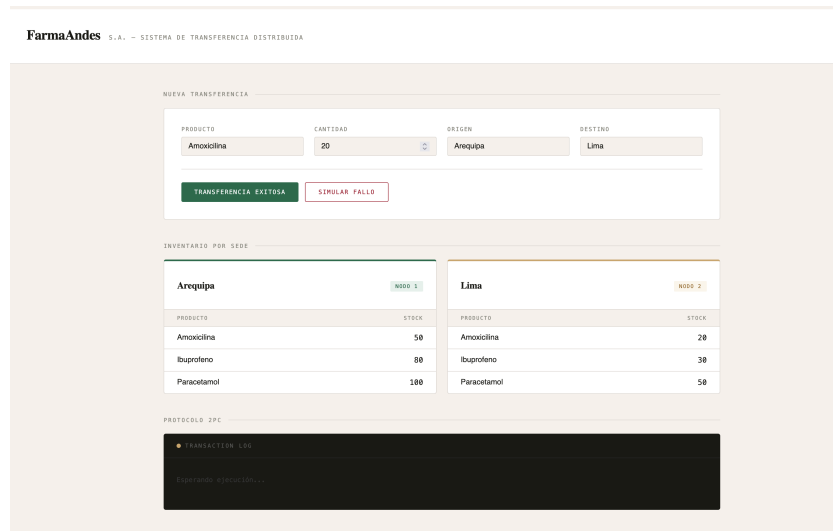


Figura 6: Interfaz web de FarmaAndes — estado inicial de los inventarios

Una vez con ambas bases de datos configuradas y la aplicación en funcionamiento, se procedió a resolver los siguientes ejercicios.

1. Transferencia Exitosa

Transferir 20 unidades de Paracetamol desde Arequipa hacia Lima.

Actividades:

1. Verificar stock disponible .
2. Iniciar transacción distribuida.
3. Descontar stock en Arequipa.
4. Incrementar stock en Lima.
5. Confirmar cambios (COMMIT global).

Implementación con 2PC:

El núcleo de la solución es la función `ejecutar_2pc` del módulo `transaccion.py`, que implementa el protocolo Two-Phase Commit sobre dos conexiones PostgreSQL. La función recibe el producto, la cantidad, las sedes origen y destino, y un flag opcional `simular_fallo`.

```
def ejecutar_2pc(
    origen_db,
    destino_db,
    producto,
    cantidad,
    simular_fallo=False
):
    """
    Protocolo Two-Phase Commit (2PC)
    - Fase 1 (PREPARE): verifica stock, descuenta en origen, pregunta a destino
    - Fase 2 (COMMIT/ROLLBACK): confirma o revierte los cambios en ambos nodos
    """

    logs = []
    # Validación inicial: origen y destino no pueden ser la misma sede
    if origen_db == destino_db:
        logs.append("ERROR: origen y destino no pueden ser iguales")
        return logs

    # Conectar a ambas bases de datos (cada una es un nodo distribuido)
    origen = connect(origen_db)
    destino = connect(destino_db)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

try:
    # --- FASE 1: PREPARE ---
    # Desactivar autocommit para tener control manual sobre la transacción
    origen.autocommit = False
    destino.autocommit = False

    cur_o = origen.cursor()
    cur_d = destino.cursor()

    nombre_origen = SEDES[origen_db]
    nombre_destino = SEDES[destino_db]

    logs.append("FASE 1: PREPARE")
    logs.append(f"Transferencia de {cantidad} unidades")
    logs.append(f"Producto: {producto}")
    logs.append(f"Origen: {nombre_origen}")
    logs.append(f"Destino: {nombre_destino}")

    # SQL: verificar stock disponible en el nodo origen
    cur_o.execute(
        """
        SELECT stock
        FROM inventario
        WHERE producto = %s
        """,
        (producto,)
    )

    fila = cur_o.fetchone()
    if fila is None:
        raise Exception("Producto no encontrado")

    stock_actual = fila[0]
    logs.append(f"Stock disponible en origen: {stock_actual}")

    # Validar que haya stock suficiente para la transferencia
    if stock_actual < cantidad:
        raise Exception("Stock insuficiente")

    # Ambos nodos responden OK en la fase PREPARE
    logs.append(f"Nodo {nombre_origen} responde OK")
    logs.append(f"Nodo {nombre_destino} responde OK")

    # SQL: descontar stock del almacén origen (UPDATE en nodo origen)
    cur_o.execute(
        """
        UPDATE inventario
        SET stock = stock - %s
        WHERE producto = %s
        """,
        (cantidad, producto)
    )

    logs.append(f"Stock descontado en {nombre_origen}")

    # START-SNIPPET,fallo_check
    # SIMULACIÓN DE FALLO (Ejercicio 2):
    # Si está activa, se lanza una excepción antes de actualizar el destino
    if simular_fallo:
        raise Exception(f"El nodo {nombre_destino} dejó de responder")

```

Antes de ejecutar la transferencia se verificó el estado inicial de los inventarios:

INVENTARIO POR SEDE																	
Arequipa NODO 1	Lima NODO 2																
<table border="1"> <thead> <tr> <th>PRODUCTO</th> <th>STOCK</th> </tr> </thead> <tbody> <tr> <td>Amoxicilina</td> <td>30</td> </tr> <tr> <td>Ibuprofeno</td> <td>80</td> </tr> <tr> <td>Paracetamol</td> <td>100</td> </tr> </tbody> </table>	PRODUCTO	STOCK	Amoxicilina	30	Ibuprofeno	80	Paracetamol	100	<table border="1"> <thead> <tr> <th>PRODUCTO</th> <th>STOCK</th> </tr> </thead> <tbody> <tr> <td>Amoxicilina</td> <td>40</td> </tr> <tr> <td>Ibuprofeno</td> <td>30</td> </tr> <tr> <td>Paracetamol</td> <td>50</td> </tr> </tbody> </table>	PRODUCTO	STOCK	Amoxicilina	40	Ibuprofeno	30	Paracetamol	50
PRODUCTO	STOCK																
Amoxicilina	30																
Ibuprofeno	80																
Paracetamol	100																
PRODUCTO	STOCK																
Amoxicilina	40																
Ibuprofeno	30																
Paracetamol	50																

Figura 7: Inventarios antes de la transferencia exitosa

Luego de ejecutar la transferencia, el log del protocolo 2PC muestra ambas fases (PREPARE y COMMIT) y los inventarios reflejan los valores actualizados:

INVENTARIO POR SEDE																	
Arequipa NODO 1	Lima NODO 2																
<table border="1"> <thead> <tr> <th>PRODUCTO</th> <th>STOCK</th> </tr> </thead> <tbody> <tr> <td>Amoxicilina</td> <td>30</td> </tr> <tr> <td>Ibuprofeno</td> <td>80</td> </tr> <tr> <td>Paracetamol</td> <td>80</td> </tr> </tbody> </table>	PRODUCTO	STOCK	Amoxicilina	30	Ibuprofeno	80	Paracetamol	80	<table border="1"> <thead> <tr> <th>PRODUCTO</th> <th>STOCK</th> </tr> </thead> <tbody> <tr> <td>Amoxicilina</td> <td>40</td> </tr> <tr> <td>Ibuprofeno</td> <td>30</td> </tr> <tr> <td>Paracetamol</td> <td>70</td> </tr> </tbody> </table>	PRODUCTO	STOCK	Amoxicilina	40	Ibuprofeno	30	Paracetamol	70
PRODUCTO	STOCK																
Amoxicilina	30																
Ibuprofeno	80																
Paracetamol	80																
PRODUCTO	STOCK																
Amoxicilina	40																
Ibuprofeno	30																
Paracetamol	70																

PROTOCOLO 2PC	
● TRANSACTION LOG » FASE 1: PREPARE » Transferencia de 20 unidades » Producto: Paracetamol » Origen: Arequipa » Destino: Lima » Stock disponible en origen: 100 » Nodo Arequipa responde OK » Nodo Lima responde OK » Stock descontado en Arequipa » Stock incrementado en Lima » Todos los nodos aceptaron la transacción » FASE 2: COMMIT » COMMIT GLOBAL EJECUTADO » Transferencia completada correctamente	

Figura 8: Resultado de la transferencia exitosa – COMMIT global ejecutado

2. Simulación de Fallo

Durante la transferencia, el nodo Lima deja de responder, lo que debe provocar un rollback global.

Actividades:

1. Iniciar transacción.
2. Descontar stock en Arequipa.
3. Simular caída de Lima.
4. Ejecutar rollback en ambos nodos.

Implementación:

La misma función `ejecutar_2pc` de `transaccion.py` (mostrada en el ejercicio anterior) soporta la simulación de fallo mediante el parámetro `simular_fallo=True`. Cuando está activo, después de descontar el stock en el origen se lanza una excepción que impide la actualización del destino y dispara el rollback.

A continuación se muestra la parte del código que detecta el fallo y la sección que ejecuta el rollback:

```
# SIMULACIÓN DE FALLO (Ejercicio 2):
# Si está activa, se lanza una excepción antes de actualizar el destino
if simular_fallo:
    raise Exception(f"El nodo {nombre_destino} dejó de responder")
```

Si la excepción se dispara, el flujo salta al bloque except, donde se ejecuta el rollback en ambas bases de datos:

```
except Exception as e:
    # Si algo falla, se ejecuta ROLLBACK en ambos nodos
    # SQL: deshacer cambios en ambas bases de datos
    origen.rollback()
    destino.rollback()

    logs.append(f"ERROR: {e}")
    logs.append("ROLLBACK GLOBAL EJECUTADO")
    logs.append("La transacción fue cancelada")
```

Esto se invoca desde la ruta /fallo de app.py, que pasa `simular_fallo=True` a `ejecutar_2pc`.

Antes de la simulación, los inventarios se encontraban en el estado posterior a la transferencia exitosa del ejercicio anterior:

INVENTARIO POR SEDE			
Arequipa		Lima	
PRODUCTO	STOCK	PRODUCTO	STOCK
Amoxicilina	30	Amoxicilina	40
Ibuprofeno	80	Ibuprofeno	30
Paracetamol	80	Paracetamol	70

Figura 9: Inventarios antes de la simulación de fallo

Al intentar la transferencia con el nodo Lima caído, el sistema ejecuta el rollback y muestra el error correspondiente:

INVENTARIO POR SEDE			
Arequipa		Lima	
PRODUCTO	STOCK	PRODUCTO	STOCK
Amoxicilina	30	Amoxicilina	40
Ibuprofeno	80	Ibuprofeno	30
Paracetamol	80	Paracetamol	70

PROTOCOLO 2PC	
- TRANSACTION LOG - FASE 1: PREPARE - Transferencia de 20 unidades - Producto: Paracetamol - Origen: Arequipa - Destino: Lima - Stock disponible en origen: 80 - Nodo Arequipa responde OK - Nodo Lima responde OK - Stock descontado en Arequipa - ERROR: El nodo Lima dejó de responder - ROLLBACK GLOBAL EJECUTADO - La transacción fue cancelada	

Figura 10: Rollback ejecutado tras la caída simulada del nodo Lima

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Caso de estudio: Sistema Nacional de Bancos Cooperativos

Una red financiera opera en tres ciudades: Arequipa, Cusco y Trujillo. Cada ciudad administra cuentas locales. Un cliente solicita transferir S/ 25 000 desde Arequipa hacia Cusco.

Restricciones:

- Debe aplicarse atomicidad.
- Debe garantizarse consistencia.
- Debe existir recuperación ante fallos.
- Debe documentarse el proceso mediante 2PC.

Implementación del caso Banco Cooperativo

Actividad 1: Diseñar el modelo distribuido.

El modelo distribuido del Sistema Nacional de Bancos Cooperativos está compuesto por tres nodos PostgreSQL independientes, cada uno alojado en una ciudad distinta: banco_arequipa, banco_cusco y banco_trujillo. La aplicación Flask actúa como orquestador, comunicándose con los tres nodos para ejecutar transacciones distribuidas bajo el protocolo 2PC. Cada nodo opera de forma autónoma y solo se sincroniza durante la transacción.

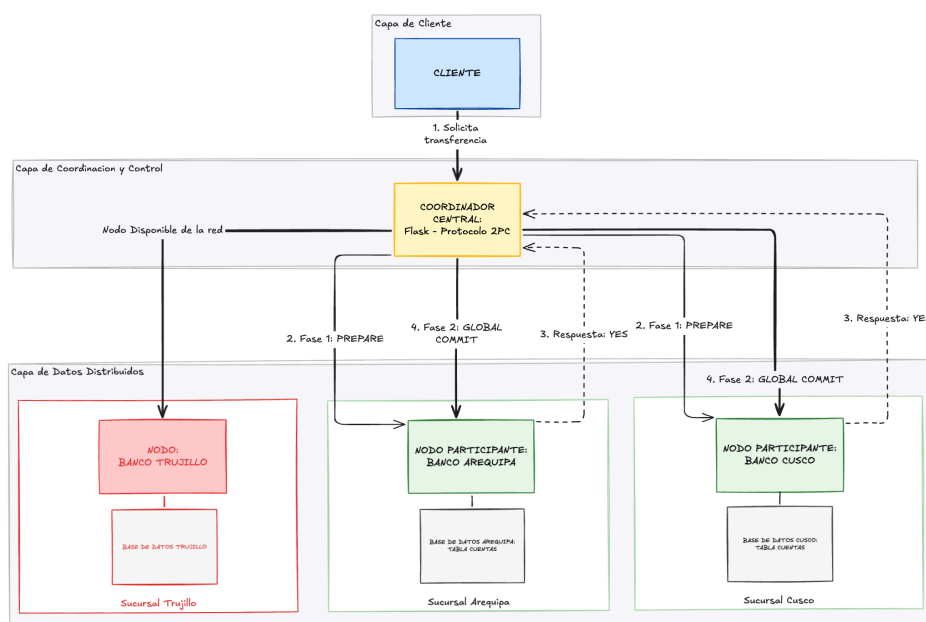


Figura 11: Diagrama de arquitectura del sistema distribuido — 3 nodos

Luego de ver la arquitectura, notaremos que para la transacción si bien son 3 nodos, solo 2 son necesarios para hacer la transacción quedando el nodo del banco de Trujillo a la espera (Nodo disponible).

Actividad 2: Identificar:

- **Coordinador:** La aplicación Flask (app.py) que ejecuta el protocolo 2PC, gestiona las fases PREPARE y COMMIT, y decide el resultado global de la transacción.
- **Participantes:** Los tres nodos PostgreSQL: banco_arequipa (origen), banco_cusco (destino) y banco_trujillo (nodo disponible).
- **Recursos involucrados:** Tabla cuentas en cada base de datos con los campos id, titular y saldo; conexiones entre la aplicación Flask y cada nodo PostgreSQL.

Actividad 3: Elaborar el diagrama de secuencia del protocolo 2PC.

El siguiente diagrama de flujo ilustra las dos fases del protocolo Two-Phase Commit: en la Fase 1 (PREPARE) el coordinador consulta a cada participante si está listo para confirmar; en la Fase 2 (COMMIT/ABORT) el coordinador notifica la decisión global.

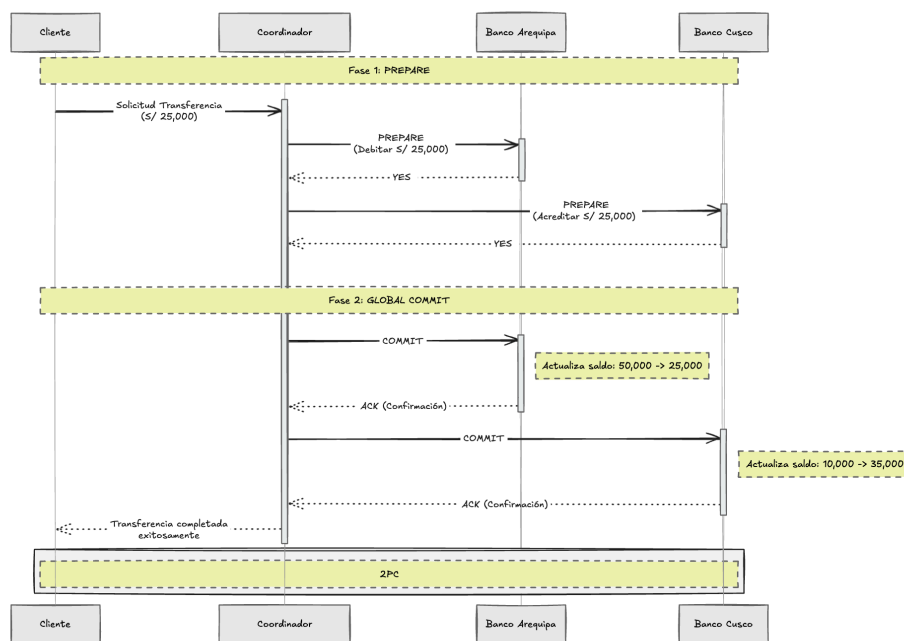


Figura 12: Diagrama de flujo del protocolo Two-Phase Commit

Actividad 4: Implementar la simulación usando PostgreSQL.

Creación de las bases de datos y tablas.

Para comenzar se crearon las bases de datos para cada sede: banco_arequipa, banco_cusco y banco_trujillo. Cada una representa un nodo PostgreSQL independiente. El siguiente comando SQL crea la base de datos y la tabla cuentas en Arequipa con sus valores iniciales.

```
CREATE TABLE cuentas(
  id serial PRIMARY KEY,
  titular VARCHAR(100),
  saldo NUMERIC(12, 2)
);

INSERT INTO cuentas(titular, saldo)
VALUES
('Alvaro Quispe', 45000);
```

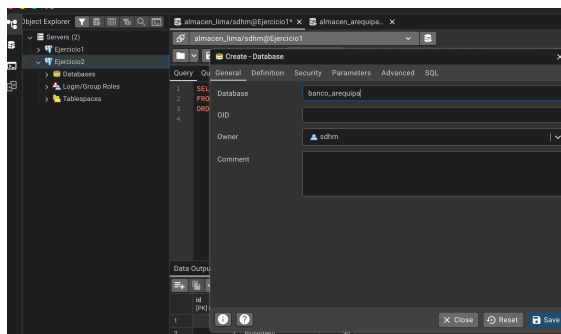


Figura 13: Creación de las bases de datos de Arequipa

La estructura de la tabla cuentas es idéntica en las tres sedes, variando únicamente el titular y el saldo inicial. Para Cusco y Trujillo se usó la misma definición SQL con otros valores.

```
CREATE TABLE cuentas(
  id serial PRIMARY KEY,
  titular VARCHAR(100),
  saldo NUMERIC(12, 2)
);

INSERT INTO cuentas(titular, saldo)
VALUES
('Fabiana Pacheco', 20000)
```

```
CREATE TABLE cuentas(
  id serial PRIMARY KEY,
  titular VARCHAR(100),
  saldo NUMERIC(12, 2)
);

INSERT INTO cuentas(titular, saldo)
VALUES
('Mathias Barrios', 30000)
```

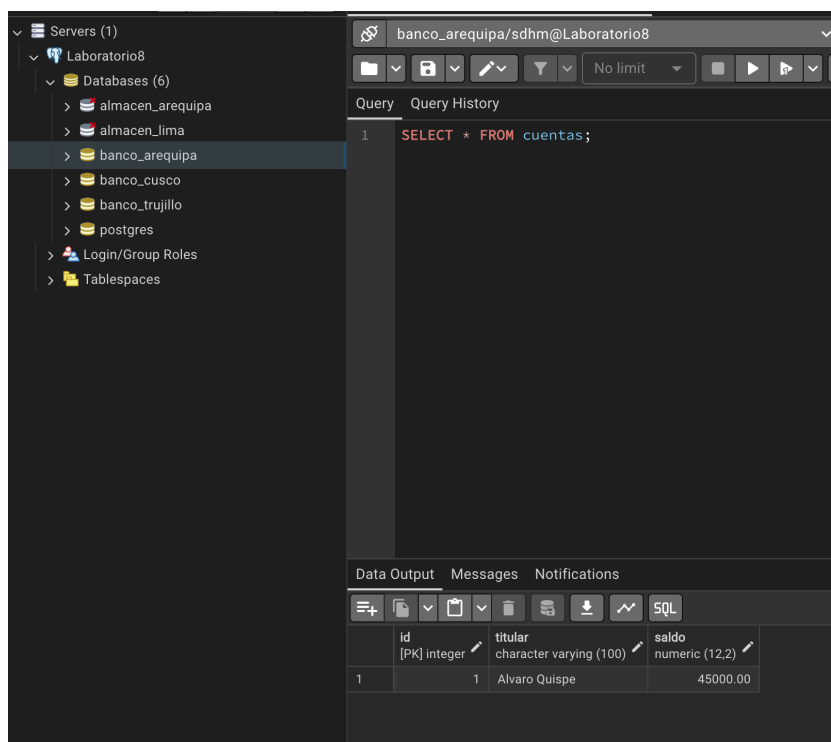


Figura 14: Verificación de datos en las tablas de los bancos con `SELECT * FROM cuentas`

Módulo transaccion.py

El archivo `transaccion.py` implementa las operaciones del protocolo 2PC. Incluye funciones auxiliares para conectar y consultar los nodos, y las 4 funciones principales del protocolo.

Funciones auxiliares

`obtener_cuentas` — consulta los saldos de todos los titulares con `SELECT titular, saldo FROM cuentas`:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```
def obtener_cuentas(nombre_db):
    conn = connect(nombre_db)
    cur = conn.cursor()
    # SELECT: titulares y saldos
    cur.execute("""
        SELECT titular, saldo
        FROM cuentas
        ORDER BY titular
    """)
    datos = cur.fetchall()
    conn.close()
    return datos
```

obtener_titulares — lista los nombres de los titulares desde Arequipa:

```
def obtener_titulares():
    conn = connect("banco_arequipa")
    cur = conn.cursor()
    # SELECT: solo nombres
    cur.execute("SELECT titular FROM cuentas ORDER BY titular")
    titulares = [fila[0] for fila in cur.fetchall()]
    conn.close()
    return titulares
```

_conectar_par — abre conexiones a dos nodos con autocommit desactivado para control manual de COMMIT y ROLLBACK:

```
def _conectar_par(origen_db, destino_db):
    origen = connect(origen_db)
    destino = connect(destino_db)
    origen.autocommit = False
    destino.autocommit = False
    return origen, destino
```

_obtener_cuenta — obtiene la primera cuenta disponible con SELECT id, titular, saldo FROM cuentas LIMIT 1:

```
def _obtener_cuenta(cur, logs, nombre):
    # SELECT: id, titular y saldo
    cur.execute("SELECT id, titular, saldo FROM cuentas LIMIT 1")
    cuenta = cur.fetchone()
    if cuenta is None:
        raise Exception(f"No existe cuenta en {nombre}")
    logs.append(f"Saldo disponible en {nombre}: S/ {cuenta[2]:.2f}")
    return cuenta
```

Funciones del protocolo 2PC

ejecutar_2pc — gestiona la transferencia exitosa con SELECT, UPDATE y COMMIT/ROLLBACK según el resultado:

```
def ejecutar_2pc(origen_db, destino_db, monto, simular_fallo=False):
    logs = []

    if origen_db == destino_db:
        logs.append("ERROR: origen y destino no pueden ser iguales")
        return logs

    nombre_origen = SEDES[origen_db]
    nombre_destino = SEDES[destino_db]

    # conectar a ambos nodos
    origen, destino = _conectar_par(origen_db, destino_db)

    try:
        cur_o = origen.cursor()
        cur_d = destino.cursor()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

# Fase 1: PREPARE
logs.append("FASE 1: PREPARE")
logs.append(f"Transferencia de S/ {monto:,.2f}")
logs.append(f"Origen: {nombre_origen} → Destino: {nombre_destino}")

# verificar saldo en origen
cuenta_origen = _obtener_cuenta(cur_o, logs, nombre_origen)
if cuenta_origen[2] < monto:
    raise Exception("Saldo insuficiente en cuenta origen")

logs.append(f"{nombre_origen} responde YES")
logs.append(f"{nombre_destino} responde YES")

# debitar monto del origen
cur_o.execute(
    "UPDATE cuentas SET saldo = saldo - %s WHERE id = %s",
    (monto, cuenta_origen[0])
)
logs.append(f"Débito realizado en {nombre_origen}")

# acreditar monto en destino
cur_d.execute(
    "UPDATE cuentas SET saldo = saldo + %s WHERE id = (SELECT id FROM cuentas LIMIT 1)",
    (monto,)
)
logs.append(f"Crédito realizado en {nombre_destino}")
logs.append("Todos los participantes votaron YES")

# Fase 2: COMMIT global
logs.append("FASE 2: GLOBAL COMMIT")

origen.commit()
destino.commit()

logs.append("COMMIT GLOBAL EJECUTADO")
logs.append("Transferencia completada exitosamente")

except Exception as e:
    # ROLLBACK si algo falla
    origen.rollback()
    destino.rollback()
    logs.append(f"ERROR: {e}")
    logs.append("GLOBAL ROLLBACK EJECUTADO")
    logs.append("Transacción cancelada – estado consistente restaurado")
finally:
    origen.close()
    destino.close()

return logs

```

`ejecutar_fallo_red` — simula un timeout en la red durante el PREPARE; el destino nunca se modifica y se ejecuta ROLLBACK:

```

def ejecutar_fallo_red(origen_db, destino_db, monto):
    logs = []

    if origen_db == destino_db:
        logs.append("ERROR: origen y destino no pueden ser iguales")
        return logs

    nombre_origen = SEDES[origen_db]
    nombre_destino = SEDES[destino_db]
    origen, destino = _conectar_par(origen_db, destino_db)

    try:
        cur_o = origen.cursor()

        # Fase 1: PREPARE
        logs.append("FASE 1: PREPARE")
        logs.append(f"Transferencia de S/ {monto:,.2f}")
        logs.append(f"Origen: {nombre_origen} → Destino: {nombre_destino}")

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

# verificar saldo en origen
cuenta_origen = _obtener_cuenta(cur_o, logs, nombre_origen)
if cuenta_origen[2] < monto:
    raise Exception("Saldo insuficiente en cuenta origen")

logs.append(f"{nombre_origen} responde YES")

# simular timeout: destino no recibe PREPARE
logs.append(f"Enviando PREPARE a {nombre_destino}...")
time.sleep(0.3)
logs.append("FALLA DE RED - No se recibió respuesta del nodo destino")
logs.append(f"Timeout: {nombre_destino} no responde (simulado)")

raise Exception(f"Falla de red - pérdida de conexión con {nombre_destino}")

except Exception as e:
    # ROLLBACK - destino nunca modificado
    origen.rollback()
    destino.rollback()
    logs.append(f"ERROR: {e}")
    logs.append("GLOBAL ROLLBACK EJECUTADO")
    logs.append("Transacción cancelada - ningún saldo fue modificado")
finally:
    origen.close()
    destino.close()

return logs

```

ejecutar_caida_nodo — el destino cae tras el débito en Fase 2; se ejecuta ROLLBACK para revertir el saldo en origen:

```

def ejecutar_caida_nodo(origen_db, destino_db, monto):
    logs = []

    if origen_db == destino_db:
        logs.append("ERROR: origen y destino no pueden ser iguales")
        return logs

    nombre_origen = SEDES[origen_db]
    nombre_destino = SEDES[destino_db]
    origen, destino = _conectar_par(origen_db, destino_db)

    try:
        cur_o = origen.cursor()

        # Fase 1: PREPARE
        logs.append("FASE 1: PREPARE")
        logs.append(f"Transferencia de S/ {monto:,.2f}")
        logs.append(f"Origen: {nombre_origen} → Destino: {nombre_destino}")

        # verificar saldo en origen
        cuenta_origen = _obtener_cuenta(cur_o, logs, nombre_origen)
        if cuenta_origen[2] < monto:
            raise Exception("Saldo insuficiente en cuenta origen")

        logs.append(f"{nombre_origen} responde YES")
        logs.append(f"{nombre_destino} responde YES")

        # debitar en origen (PREPARE exitoso)
        cur_o.execute(
            "UPDATE cuentas SET saldo = saldo - %s WHERE id = %s",
            (monto, cuenta_origen[0])
        )
        logs.append(f"Débito realizado en {nombre_origen}")
        logs.append("FASE 2: GLOBAL COMMIT iniciado")
        logs.append(f"Enviando COMMIT a {nombre_destino}...")

        # simular caída del destino en Fase 2
        logs.append(f"CAÍDA DE NODO - {nombre_destino} dejó de responder")
        logs.append(f"El proceso en {nombre_destino} fue terminado (simulado)")
    
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

        raise Exception(f" Nodo caído: {nombre_destino} no completó el COMMIT")

except Exception as e:
    # ROLLBACK – revertir débito en origen
    origen.rollback()
    destino.rollback()
    logs.append(f"ERROR: {e}")
    logs.append("GLOBAL ROLLBACK EJECUTADO")
    logs.append("Débito revertido en origen – estado consistente restaurado")
finally:
    origen.close()
    destino.close()

return logs

```

ejecutar_recuperacion — reintenta la transacción completa una vez que ambos nodos están en línea:

```

def ejecutar_recuperacion(origen_db, destino_db, monto, tipo_fallo):
    logs = []

    if origen_db == destino_db:
        logs.append("ERROR: origen y destino no pueden ser iguales")
        return logs

    nombre_origen = SEDES[origen_db]
    nombre_destino = SEDES[destino_db]
    origen, destino = _conectar_par(origen_db, destino_db)

    try:
        cur_o = origen.cursor()
        cur_d = destino.cursor()

        logs.append("RECUPERACIÓN POSTERIOR")
        logs.append("Verificando estado de nodos tras fallo previo...")
        logs.append(f" Nodo {nombre_origen}: en línea ✓")
        logs.append(f" Nodo {nombre_destino}: en línea ✓ (recuperado)")
        logs.append("Consultando log de transacciones pendientes...")

        logs.append(f"RECUPERACIÓN – Resolviendo fallo de tipo: {tipo_fallo.replace('_', ' ').upper()}")

        # Fase 1: PREPARE (reintento)
        logs.append("FASE 1: PREPARE (reintento)")

        # verificar saldo actual
        cuenta_origen = _obtener_cuenta(cur_o, logs, nombre_origen)

        logs.append(f"{nombre_origen} (Participante 1) – Vota: YES")
        logs.append(f"{nombre_destino} (Participante 2) – Vota: YES (Recuperado)")

        # UPDATE: debitar y acreditar pendientes
        cur_o.execute("UPDATE cuentas SET saldo = saldo - %s WHERE id = %s", (monto, cuenta_origen[0]))
        cur_d.execute("UPDATE cuentas SET saldo = saldo + %s WHERE id = (SELECT id FROM cuentas LIMIT 1)", (monto,))

        # Fase 2: COMMIT global
        logs.append("FASE 2: GLOBAL COMMIT")
        origen.commit()
        destino.commit()

        logs.append("LOG: Transacción marcada como COMPLETADA en el Coordinador")
        logs.append("RECUPERACIÓN EXITOSA – Consistencia total restaurada en todos los nodos")

    except Exception as e:
        origen.rollback()
        destino.rollback()
        logs.append(f"ERROR durante recuperación: {e}")
        logs.append("GLOBAL ROLLBACK EJECUTADO")
        logs.append("Recuperación fallida – intervención manual requerida")
    finally:
        origen.close()
        destino.close()

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```
return logs
```

Coordinador Flask

La aplicación app.py expone rutas HTTP que invocan las funciones anteriores. Cada ruta se comenta con las operaciones SQL que desencadena:

```
# Módulo de conexión a PostgreSQL – Sistema Nacional de Bancos Cooperativos
# Cada "nodo" es una base de datos PostgreSQL independiente

import psycopg2

def connect(dbname):
    """Conecta a una base de datos PostgreSQL local (cada sede es un nodo)"""
    return psycopg2.connect(
        dbname=dbname,
        user="sdhm",
        password="",
        host="localhost",
        port="5432"
    )
```

```
# Aplicación web Flask – Sistema Nacional de Bancos Cooperativos
# Interfaz para transferencias distribuidas con 2PC y simulación de fallos

from flask import Flask, render_template, request, session
from transaccion import (
    SEDES,
    obtener_cuentas,
    ejecutar_2pc,
    ejecutar_fallo_red,
    ejecutar_caída_nodo,
    ejecutar_recuperacion
)

app = Flask(__name__, template_folder="../templates")
app.secret_key = "bancos_cooperativos_2pc"

def renderizar_inicio(logs=None):
    if logs is None:
        logs = []
    return render_template(
        "index.html",
        banco_arequipa=obtener_cuentas("banco_arequipa"),
        banco_cusco=obtener_cuentas("banco_cusco"),
        banco_trujillo=obtener_cuentas("banco_trujillo"),
        sedes=SEDES,
        logs=logs,
        ultimo_fallo=session.get("ultimo_fallo")
    )

# Muestra los saldos iniciales de los 3 bancos (SELECT en cada nodo)
@app.route("/")
def index():
    return renderizar_inicio()

# Ejecuta 2PC: SELECT saldo → UPDATE débito → UPDATE crédito → COMMIT/ROLLBACK
@app.route("/transferir", methods=["POST"])
def transferir():
    origen = request.form["origen"]
    destino = request.form["destino"]
    monto = float(request.form["monto"])
    logs = ejecutar_2pc(origen, destino, monto)
    session.pop("ultimo_fallo", None)
    return renderizar_inicio(logs)
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

```
# Falla de red: SELECT saldo → timeout simulado → ROLLBACK
@app.route("/fallo_red", methods=["POST"])
def fallo_red():
    origen = request.form["origen"]
    destino = request.form["destino"]
    monto = float(request.form["monto"])
    logs = ejecutar_fallo_red(origen, destino, monto)
    session["ultimo_fallo"] = {
        "tipo": "fallo_red",
        "origen": origen,
        "destino": destino,
        "monto": monto
    }
    return renderizar_inicio(logs)

# Caída de nodo: SELECT → UPDATE débito → nodo cae → ROLLBACK
@app.route("/caida_nodo", methods=["POST"])
def caida_nodo():
    origen = request.form["origen"]
    destino = request.form["destino"]
    monto = float(request.form["monto"])
    logs = ejecutar_caida_nodo(origen, destino, monto)
    session["ultimo_fallo"] = {
        "tipo": "caida_nodo",
        "origen": origen,
        "destino": destino,
        "monto": monto
    }
    return renderizar_inicio(logs)

# Recuperación: SELECT → UPDATE débito/crédito → COMMIT (reintento completo)
@app.route("/recuperacion", methods=["POST"])
def recuperacion():
    fallo = session.get("ultimo_fallo")

    if not fallo:
        logs = [
            "RECUPERACION - Sin fallo previo registrado.",
            "No hay ningun nodo caido ni falla de red activa.",
            "Ejecuta primero Fallo de Red o Caída de Nodo para simular un fallo."
        ]
        return renderizar_inicio(logs)

    logs = ejecutar_recuperacion(
        fallo["origen"],
        fallo["destino"],
        fallo["monto"],
        fallo["tipo"]
    )
    session.pop("ultimo_fallo", None)
    return renderizar_inicio(logs)

if __name__ == "__main__":
    app.run(debug=True)
```



```

C:\SD\labSQL\casoBancoCooperativo
> python3 python/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 141-887-684

```

Figura 15: Ejecución de la aplicación Flask — coordinador del sistema distribuido

Actividad 5: Simular.

5.1 Transferencia Exitosa

Transferir S/ 25 000 desde la cuenta de Alvaro Quispe (Arequipa) hacia la cuenta de Fabiana Pacheco (Cusco) mediante el protocolo 2PC. (Ver detalle del código en Actividad 4, función ejecutar_2pc.)

Actividades:

1. Verificar saldo disponible.
2. Iniciar transacción distribuida (Fase 1: PREPARE).
3. Debitar S/ 25 000 en Arequipa.
4. Acreditar S/ 25 000 en Cusco.
5. Confirmar cambios (Fase 2: COMMIT global).

Antes de ejecutar la transferencia se verificó el estado inicial de los saldos:

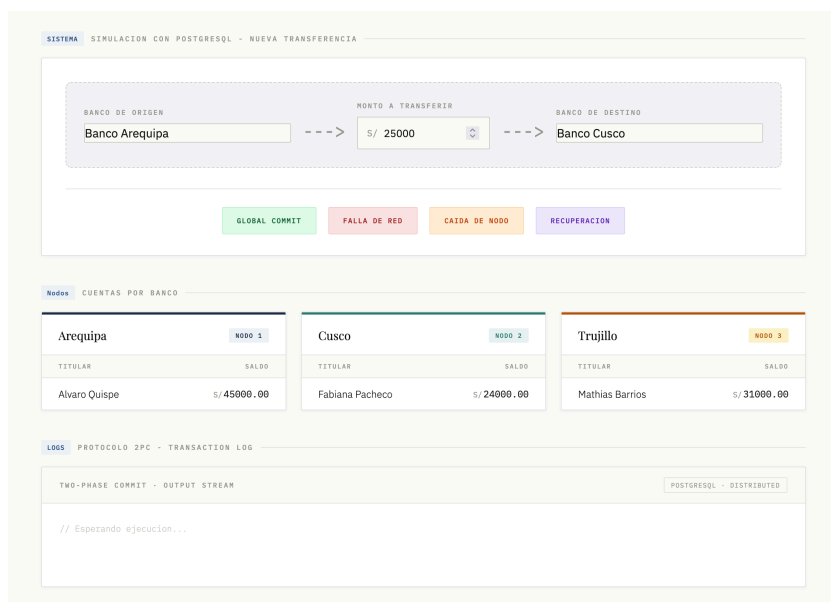
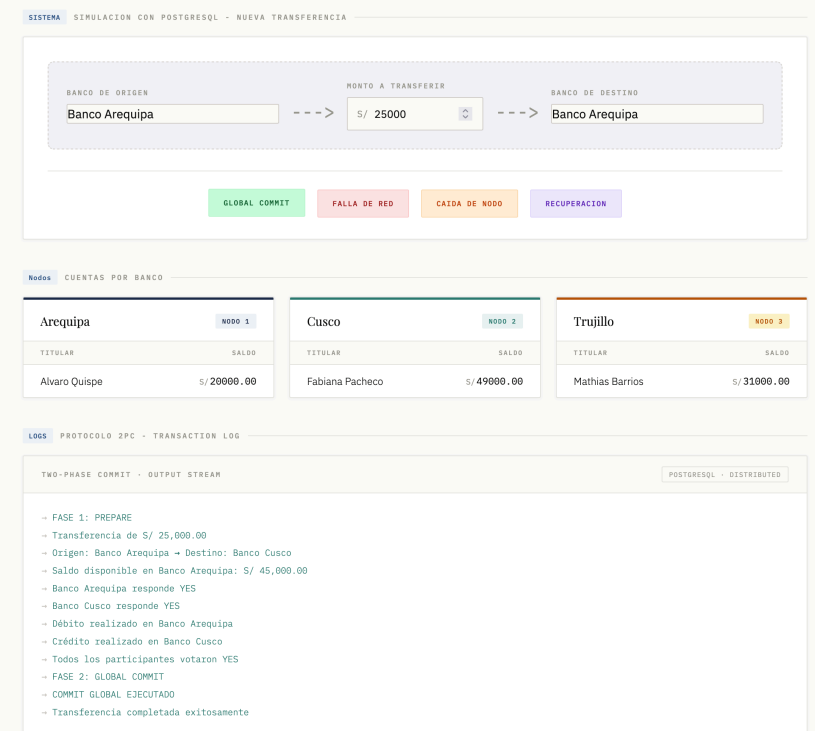


Figura 16: Saldos antes de la transferencia exitosa

Luego de ejecutar la transferencia, el log del protocolo 2PC muestra ambas fases (PREPARE y COMMIT) y los saldos reflejan los valores actualizados:



SISTEMA SIMULACION CON POSTGRESQL - NUEVA TRANSFERENCIA

BANCO DE ORIGEN: Banco Arequipa
 --> MONTO A TRANSFERIR: S/ 25000
 --> BANCO DE DESTINO: Banco Arequipa

GLOBAL COMMIT
FALLA DE RED
CAIDA DE NODO
RECUPERACION

Nodos CUENTAS POR BANCO

Arequipa NODO 1		Cusco NODO 2		Trujillo NODO 3	
TITULAR	SALDO	TITULAR	SALDO	TITULAR	SALDO
Alvaro Quispe	S/ 20000.00	Fabiana Pacheco	S/ 49000.00	Mathias Barrios	S/ 31000.00

LOGS PROTOCOLO 2PC - TRANSACTION LOG

TWO-PHASE COMMIT - OUTPUT STREAM POSTGRESQL - DISTRIBUTED

```

-> FASE 1: PREPARE
-> Transferencia de S/ 25,000.00
-> Origen: Banco Arequipa -> Destino: Banco Cusco
-> Saldo disponible en Banco Arequipa: S/ 45,000.00
-> Banco Arequipa responde YES
-> Banco Cusco responde YES
-> Débito realizado en Banco Arequipa
-> Crédito realizado en Banco Cusco
-> Todos los participantes votaron YES
-> FASE 2: GLOBAL COMMIT
-> COMMIT GLOBAL EJECUTADO
-> Transferencia completada exitosamente
          
```

Figura 17: Resultado de la transferencia exitosa — COMMIT global ejecutado

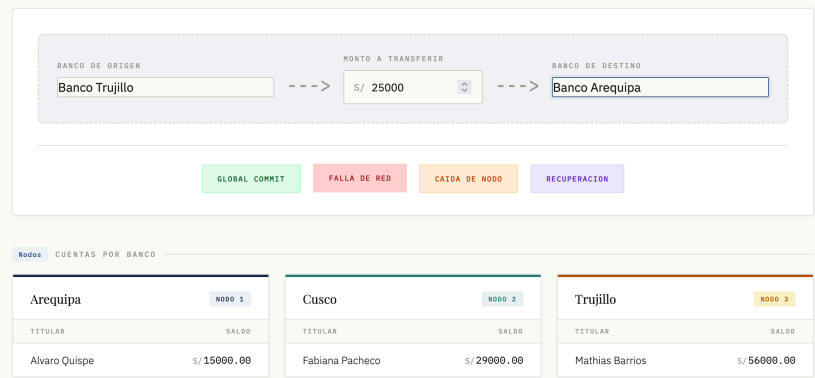
5.2 Simulación de Falla de Red

Durante la Fase 1 (PREPARE), el nodo destino (Cusco) no responde por una falla de red simulada. El coordinador detecta el timeout y ejecuta rollback sin haber modificado ningún saldo. (Ver detalle del código en Actividad 4, función ejecutar_fallo_red.)

Actividades:

1. Iniciar transacción.
2. Verificar saldo en origen (Arequipa).
3. Simular timeout de red hacia Cusco.
4. Ejecutar rollback — ningún saldo fue modificado.

Antes de la simulación, los saldos se encontraban en el estado posterior a la transferencia exitosa del ejercicio anterior:



SISTEMA SIMULACION CON POSTGRESQL - NUEVA TRANSFERENCIA

BANCO DE ORIGEN: Banco Trujillo
 --> MONTO A TRANSFERIR: S/ 25000
 --> BANCO DE DESTINO: Banco Arequipa

GLOBAL COMMIT
FALLA DE RED
CAIDA DE NODO
RECUPERACION

Nodos CUENTAS POR BANCO

Arequipa NODO 1		Cusco NODO 2		Trujillo NODO 3	
TITULAR	SALDO	TITULAR	SALDO	TITULAR	SALDO
Alvaro Quispe	S/ 15000.00	Fabiana Pacheco	S/ 29000.00	Mathias Barrios	S/ 56000.00

Figura 18: Saldos antes de la simulación de falla de red

Al intentar la transferencia con la red caída, el sistema aborta la operación sin modificar ningún saldo:

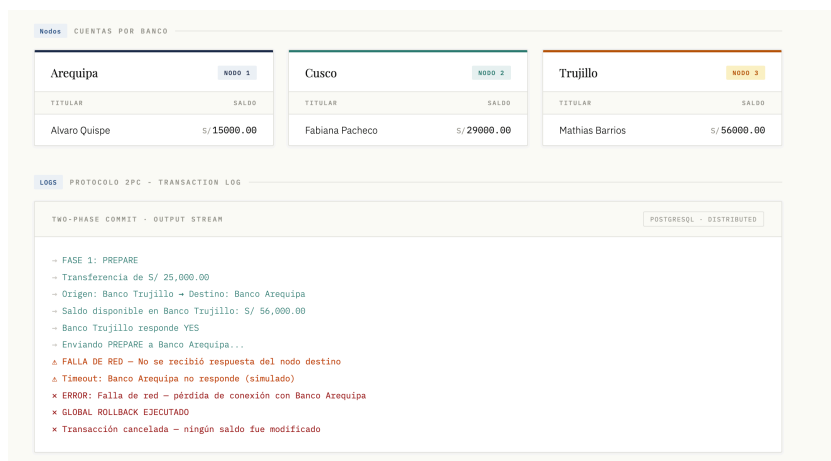


Figura 19: Rollback ejecutado tras falla de red — ningún saldo modificado

5.3 Simulación de Caída de Nodo

Durante la Fase 2 (COMMIT), después de haber debitado en origen, el nodo destino (Cusco) deja de responder. El coordinador ejecuta rollback para revertir el débito y mantener la consistencia. (Ver detalle del código en Actividad 4, función ejecutar_caída_nodo.)

Actividades:

1. Iniciar transacción.
2. Verificar saldo en origen.
3. Debitar en Arequipa (PREPARE exitoso).
4. Simular caída de Cusco durante el COMMIT.
5. Ejecutar rollback — débito revertido.

Antes de la simulación, los saldos estaban en el estado posterior a la transferencia exitosa:

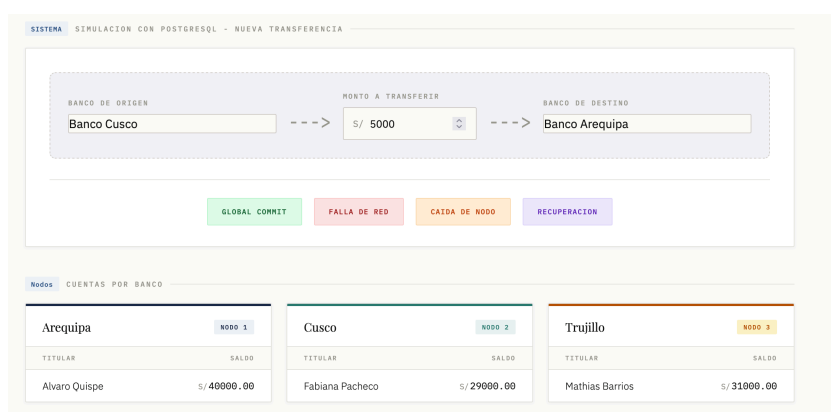


Figura 20: Saldos antes de la simulación de caída de nodo

Al caer el nodo destino, el sistema revierte el débito en origen:

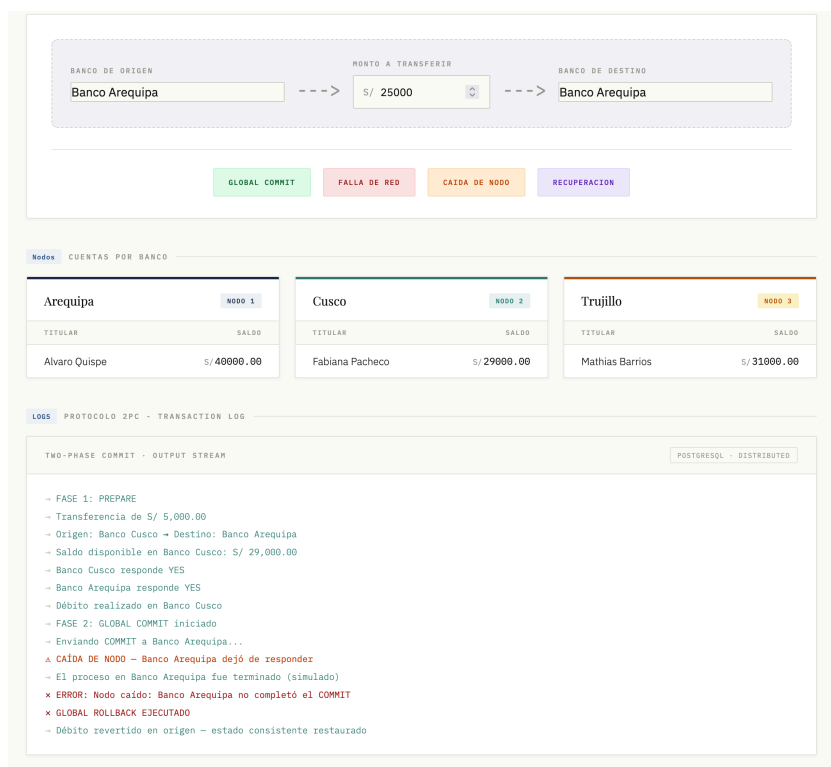


Figura 21: Rollback ejecutado tras caída de nodo — débito revertido

5.4 Recuperación Posterior

Luego de una falla (de red o de nodo), el sistema puede recuperarse re-ejecutando la transacción pendiente. Se verifica que ambos nodos estén en línea y se completa la operación. (Ver detalle del código en Actividad 4, función ejecutar_recuperacion.)

Actividades:

1. Verificar estado de los nodos (origen y destino en línea).
2. Consultar log de transacciones pendientes.
3. Re-ejecutar PREPARE (Fase 1).
4. Ejecutar COMMIT global (Fase 2).

Recuperación tras una falla de red — fallo original:

Nodos

Cuentas por Banco

Arequipa

Nodo 1

TITULAR

SALDO

Alvaro Quispe

S/ 15000.00

Cusco

Nodo 2

TITULAR

SALDO

Fabiana Pacheco

S/ 29000.00

Trujillo

Nodo 3

TITULAR

SALDO

Mathias Barrios

S/ 56000.00

LOGS

PROTOCOLO 2PC - TRANSACTION LOG

TWO-PHASE COMMIT - OUTPUT STREAM

POSTGRESQL - DISTRIBUTED

- FASE 1: PREPARE

- Transferencia de S/ 25,000.00

- Origen: Banco Trujillo - Destino: Banco Arequipa

- Saldo disponible en Banco Trujillo: S/ 56,000.00

- Banco Trujillo responde YES

- Enviando PREPARE a Banco Arequipa...

Δ FALLA DE RED - No se recibió respuesta del nodo destino

Δ Timeout: Banco Arequipa no responde (simulado)

× ERROR: Falla de red - pérdida de conexión con Banco Arequipa

× GLOBAL ROLLBACK EJECUTADO

× Transacción cancelada - ningún saldo fue modificado

Figura 22: Falla de red original — rollback ejecutado, saldos intactos

Una vez restaurada la conectividad, el coordinador re-ejecuta la transacción pendiente con éxito:

Nodos

Cuentas por Banco

Arequipa

NODO 1

TITULAR	SALDO
Alvaro Quispe	S/ 40000.00

Cusco

NODO 2

TITULAR	SALDO
Fabiana Pacheco	S/ 29000.00

Trujillo

NODO 3

TITULAR	SALDO
Mathias Barrios	S/ 31000.00

LOGS

PROTOCOLO 2PC - TRANSACTION LOG

TWO-PHASE COMMIT - OUTPUT STREAM

POSTGRESQL - DISTRIBUTED

✓ RECUPERACIÓN POSTERIOR

→ Verificando estado de nodos tras fallo previo...

✓ Nodo Banco Trujillo: en línea ✓

✓ Nodo Banco Arequipa: en línea ✓ (recuperado)

→ Consultando log de transacciones pendientes...

✓ RECUPERACIÓN – Resolviendo fallo de tipo: FALLO RED

→ FASE 1: PREPARE (reintento)

→ Saldo disponible en Banco Trujillo: S/ 56,000.00

→ Banco Trujillo (Participante 1) – Vota: YES

✓ Banco Arequipa (Participante 2) – Vota: YES (Recuperado)

→ FASE 2: GLOBAL COMMIT

→ LOG: Transacción marcada como COMPLETADA en el Coordinador

✓ RECUPERACIÓN EXITOSA – Consistencia total restaurada en todos los nodos

Figura 23: Recuperación exitosa luego de una falla de red

Recuperación tras una caída de nodo — fallo original:

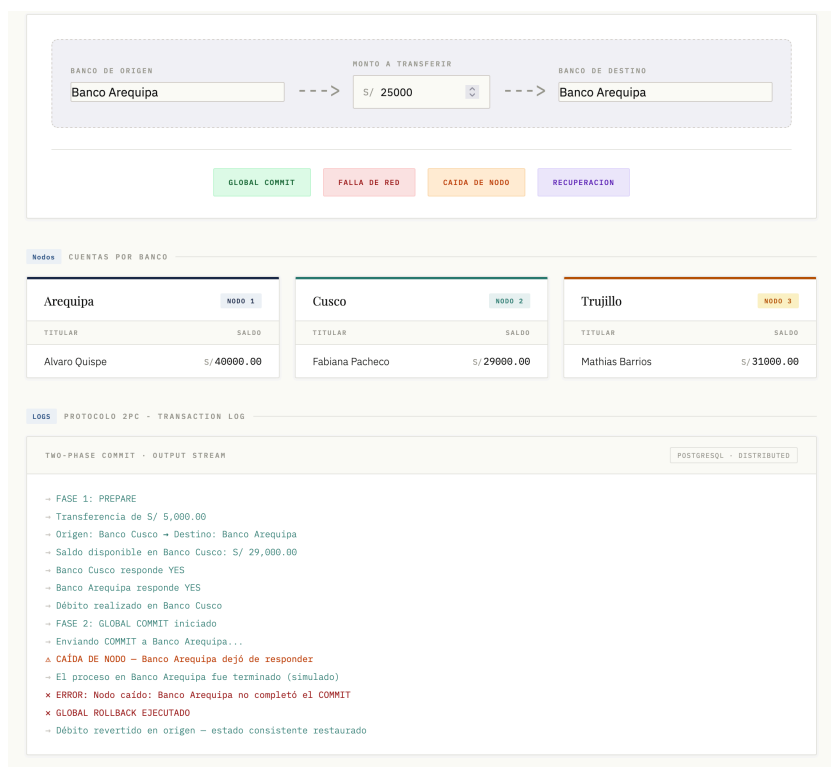


Figura 24: Caída de nodo original — débito revertido, estado consistente

Tras reiniciar el nodo caído, el coordinador completa la transacción:

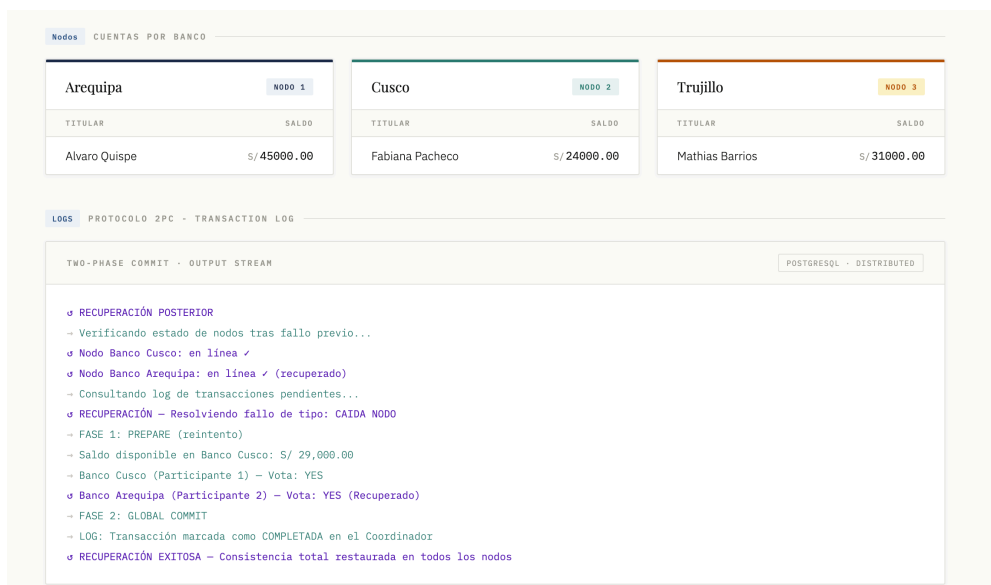


Figura 25: Recuperación exitosa luego de una caída de nodo

Actividad 6: Analizar.

La siguiente tabla resume el impacto del protocolo 2PC sobre las dimensiones críticas del sistema y las estrategias aplicadas.

Dimensión	Impacto observado	Estrategias aplicadas
Consistencia	El protocolo 2PC garantiza consistencia fuerte: todos los nodos reflejan el mismo estado al final. Sin embargo, durante la ventana entre PREPARE y COMMIT los recursos quedan bloqueados.	- Log de transacciones persistente en cada nodo - COMMIT en dos fases para evitar escrituras parciales - Rollback automático ante fallo detectado
Disponibilidad	El coordinador es punto único de fallo. Si falla durante la Fase 2, los participantes quedan en estado incierto hasta su recuperación. Una caída de nodo detiene las operaciones del sistema.	- Coordinador con failover manual - Timeouts configurables (0.3 s) para detectar fallos de red - Recuperación posterior re-ejecutando la transacción pendiente
Recuperación	El sistema se recupera re-ejecutando la transacción desde el log. Si el log no es persistente, se pierde el contexto del fallo y se requiere intervención manual.	- Función ejecutar_recuperacion con reintento completo del 2PC - Verificación de estado de nodos antes de reintentar - Limpieza del estado de fallo tras recuperación exitosa

CUESTIONARIO

II. CUESTIONARIO

1. Una empresa financiera prioriza la disponibilidad del servicio sobre la consistencia de los datos. ¿Qué riesgos podrían surgir y cómo afectarían a los clientes?

Una empresa financiera que prioriza la disponibilidad sobre la consistencia (modelo AP del teorema CAP [1]) expone los datos a lecturas inconsistentes. Por ejemplo, un cliente consulta su saldo justo después de un retiro y aún ve el monto anterior, o una transferencia entre cuentas se registra en una pero no en la otra.

Riesgos concretos: pagos duplicados, sobregiros no detectados, registros contables desincronizados. Los clientes pueden creer que tienen fondos que ya no están disponibles, generando rechazos en transacciones posteriores, cargos por sobregiro inesperados o pérdida de confianza en la entidad.

2. El protocolo Two-Phase Commit garantiza consistencia, pero puede reducir la disponibilidad del sistema. ¿Considera que este sacrificio es justificable en todos los contextos empresariales? Fundamente su respuesta.

Two-Phase Commit (2PC [2]) garantiza que todos los nodos confirmen o aborten una transacción atómicamente, pero bloquea recursos durante la Coordinación. Si el coordinador falla, los nodos quedan bloqueados hasta que se recupere, reduciendo la disponibilidad.

Este sacrificio *no* es justificable en todos los contextos. Es aceptable en sistemas financieros donde una transferencia bancaria debe ser exacta (consistencia fuerte). Sin embargo, en redes sociales o e-commerce, una transacción parcial (consistencia eventual) es preferible a denegar el servicio. Ejemplo: si un banco usa 2PC y el coordinador cae, el cajero automático no puede procesar retiros; un sistema AP simplemente mostraría “saldo actualizado en breve” y permitiría la operación.

3. Imagine que una organización global opera cientos de nodos distribuidos. ¿Qué alternativas al protocolo 2PC podrían mejorar el rendimiento sin comprometer significativamente la confiabilidad del sistema?

Existen alternativas al 2PC que mejoran el rendimiento en entornos con cientos de nodos:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 25

- **Saga Pattern [3]:** divide la transacción en pasos locales con eventos compensatorios. Si un paso falla, se ejecutan las compensaciones de los pasos anteriores. No bloquea recursos, pero requiere lógica de reversión. Ideal para microservicios.
- **Consenso distribuido (Raft [4] / Paxos [5]):** los nodos acuerdan un orden de operaciones sin un coordinador central frágil. Soportan fallos de hasta minority de nodos sin perder disponibilidad.
- **Base de datos distribuidas con consistencia eventual (Cassandra, DynamoDB [6]):** usan replicación asíncrona y resolución de conflictos mediante vectores de versiones. Sacrifican consistencia inmediata por alta disponibilidad y escalabilidad lineal.
- **Protocolo TCC (Try-Confirm/Cancel):** similar a Saga, pero cada recurso reserva el cambio en una fase Try y luego Confirma o Cancela. Usado en sistemas de pago donde se necesita consistencia sin bloqueos largos.

CONCLUSIONES

III. CONCLUSIONES

El equilibrio entre consistencia y disponibilidad no es técnico, es humano [7]. Los sistemas financieros que priorizan la consistencia (CP) protegen al usuario de errores económicos, mientras que los que priorizan la disponibilidad (AP) lo protegen de la frustración de un servicio caído. No hay protocolo universal: 2PC ofrece fiabilidad pero centraliza el riesgo en el coordinador; Sagas y TCC delegan la responsabilidad en compensaciones locales, siendo más ágiles pero más complejos de diseñar. Elegir entre ellos es decidir qué error estamos dispuestos a que el usuario experimente.

La tecnología distribuida avanza hacia modelos más resilientes, pero ningún protocolo reemplaza el criterio de dominio. Mientras 2PC y Raft garantizan un estado global predecible, las arquitecturas basadas en eventos (Sagas, DynamoDB) abrazan la incertidumbre y la resuelven después. La decisión final no es técnica: es entender que detrás de cada transacción hay una persona esperando una respuesta, y el sistema debe estar diseñado para no fallarle dos veces.

REFERENCIAS

Bibliografía

- [1] E. A. Brewer, "Towards Robust Distributed Systems," in *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 2000, pp. 7–10.
- [2] P. A. Bernstein, V. Hadzilacos, and N. Goodman, "Concurrency Control and Recovery in Database Systems," Addison-Wesley, technical report, 1987.
- [3] H. Garcia-Molina and K. Salem, "Sagas," *ACM SIGMOD Record*, vol. 16, no. 3, pp. 249–259, 1987.
- [4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014, pp. 305–319.
- [5] L. Lamport, "Paxos Made Simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, 2001.
- [6] G. DeCandia and others, "Dynamo: Amazon's Highly Available Key-value Store," in *Proceedings of 21st ACM SIGOPS Symposium on Operating Systems Principles*, 2007, pp. 205–220.
- [7] A. S. Tanenbaum and M. V. Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Pearson, 2007.